Федеральное государственное автономное образовательное учреждение
высшего образования
**«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»**

ОТЧЕТ

по лабораторной работе № 2
по дисциплине «Основы программной инженерии»

Автоматизация технологического процесса промышленного производства программного обеспечения

Вариант 34

«Разработка программного обеспечения для автоматизированной/ информационной системы фирмы по сборке и продаже компьютеров и комплектующих»

Выполнили:
Кочнев Д.О., Кочнев С.О., Королёв И.М.
Группа:
4431

Преподаватель:
_____

Дата защиты:
_____

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ....4

ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ № 2....5

ВВЕДЕНИЕ....6

1 Анализ предметной области и постановка задачи....8

1.1 Предметная область и участники....8

1.2 Функциональные требования....9

1.3 Нефункциональные требования и ограничения....10

1.4 Постановка задачи....11

2 Этап I. Организация технологического процесса....12

2.1 Виртуальная машина и среда разработки....12

2.2 Установка и настройка Redmine....13

2.3 Пользователи, роли и права....13

2.4 Трекеры, статусы и переходы....15

2.5 Дерево целей и задач....17

2.6 Репозиторий и Git в виртуальной машине....18

3 Этап II. Реализация программного обеспечения....20

3.1 Архитектура и технологический стек....20

3.2 Модель данных....21

3.3 Бизнес-алгоритмы и жизненный цикл заказа....23

3.4 Внешние интеграции, безопасность и развёртывание....26

4 Тестирование и результаты....28

4.1 Методика и окружение....28

4.2 Модульные и приёмочные тесты....28

4.3 Кроссбраузерные сценарии....29

4.4 Проверка модели работ в Redmine....34

4.5 Соответствие нефункциональным требованиям....35

5 Применение Git и модели Git Flow....36

5.1 Ветвление и история репозитория....36

5.2 Интеграция этапа II....37

ВЫВОДЫ..... 38
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....40
ПРИЛОЖЕНИЕ А. Основные команды и структура проекта.....41

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

API — Application Programming Interface — программный интерфейс приложения.

CSP — Content Security Policy — политика безопасности содержимого.

HMAC — Hash-based Message Authentication Code — код аутентификации сообщения.

JWT — JSON Web Token — веб-токен в формате JSON.

NFR — Non-Functional Requirement — нефункциональное требование.

RBAC — Role-Based Access Control — управление доступом на основе ролей.

SPA — Single Page Application — одностраничное web-приложение.

TLS — Transport Layer Security — протокол защиты транспортного уровня.

БД — база данных.

ВМ — виртуальная машина.

ИС — информационная система.

ПК — персональный компьютер.

FR — Functional Requirement — функциональное требование.

ЗАДАНИЕ НА ЛАБОРАТОРНУЮ РАБОТУ № 2

Дисциплина: «Основы программной инженерии». Группа: 4431.

Тема: автоматизация технологического процесса промышленного производства программного обеспечения. Вариант 34: разработка программного обеспечения для автоматизированной/информационной системы фирмы по сборке и продаже компьютеров и комплектующих.

Этап I. Автоматизация технологического процесса промышленного производства программного обеспечения. Установка и настройка виртуальной машины для разработки. Развертывание, конфигурирование и применение систем управления проектами и контроля версий. Постановка задач и распределение зон ответственности в проекте. Коллективное взаимодействие в группе из трех человек в ролях «Руководитель проектов в области информационных технологий», «Специалист по информационным системам», «DevOps-инженер».

Этап II. Разработка программного обеспечения в соответствии с требованиями, предъявленными в первой лабораторной работе и поставленными задачами на первом этапе. Коллективное взаимодействие в группе из трех человек в ролях «Системный программист», «Программист», «Администратор баз данных».

Задание выдал: _____ / _____ /

«___» _____ 2026 г.

Задание приняли:

_____ / Кочнев С.О. /

_____ / Кочнев Д.О. /

_____ / Королёв И.М. /

ВВЕДЕНИЕ

Современная фирма по сборке и продаже компьютеров одновременно работает с каталогом комплектующих, складскими остатками, индивидуальными конфигурациями, заказами, оплатой, сборкой, тестированием и доставкой. При частичной автоматизации эти процессы ведутся в разрозненных таблицах и сообщениях, поэтому цена, наличие и состояние заказа могут расходиться. Ошибка в совместимости компонентов или резервирование отсутствующего товара приводит к повторным согласованиям, задержкам и финансовым потерям.

Цель лабораторной работы — сформировать практические навыки организации технологического процесса промышленного производства программного обеспечения. Для достижения цели необходимо выполнить оба этапа методических указаний: развернуть виртуальную машину и систему управления проектами, сформировать модель работ и ролевую модель, применить систему контроля версий, а затем реализовать и проверить информационную систему по требованиям первой лабораторной работы.

В ходе работы решались следующие задачи: развертывание изолированной виртуальной среды; установка и конфигурирование Redmine; создание проекта, учетных записей, ролей, трекеров, статусов и переходов; построение дерева целей и задач по требованиям FR-01–FR-22; настройка Git и модели Git Flow; разработка серверной и клиентской частей ИС; реализация транзакционного оформления заказа, оплаты, складского учета, сборки и доставки; автоматическое тестирование; оформление отчета и доклада.

Работа выполнялась командой из трех человек. Распределение ролей этапа I и зон ответственности этапа II приведено в таблице 1. Этап II выполнялся той же командой; области ответственности соответствуют серверной части, клиентской части и базе данных.

Таблица 1 – Роли команды и зоны ответственности

Роль этапа I	Участник	Зона ответственности этапа I	Зона ответственности этапа II
Руководитель проектов в области ИТ	Кочнев С.О.	BM, Redmine, проект, пользователи, роли, трекеры, статусы, дерево задач, Git, отчет	Системный программист: серверная часть, API, интеграции, тесты
Специалист по информационным системам	Кочнев Д.О.	Детальная установка Redmine, тема Bleuclair, Git, исходные файлы и README	Программист: клиентская часть React/TypeScript, сценарии интерфейса
DevOps-инженер	Королёв И.М.	Git, четыре учетные записи, репозиторий, модель Git Flow	Администратор баз данных: схема PostgreSQL, миграции, резервное копирование, контейнеры

В методических указаниях 2021 года для этапа I описаны Oracle VirtualBox, Astra Linux, Redmine 4.0.5, PostgreSQL 9.6 и Apache/Passenger. В данной работе использованы QEMU/KVM, Ubuntu 24.04, Redmine 6.1.5, PostgreSQL 16 и nginx/Puma. Такая замена обеспечивает воспроизводимость в текущей учебной среде, но не была предварительно согласована с преподавателем; это ограничение зафиксировано в выводах и должно быть учтено при защите.

1 Анализ предметной области и постановка задачи

1.1 Предметная область и участники

Предметная область охватывает продажу компьютерных комплектующих и услугу индивидуальной сборки ПК. Покупатель выбирает готовые товары или формирует конфигурацию, оформляет заказ, оплачивает его и отслеживает состояние. Менеджер ведет каталог и обрабатывает заказы, сотрудник склада отвечает за остатки и комплектацию, сборщик — за сборку и тестирование, администратор — за учетные записи и права.

Ключевыми объектами предметной области являются товар, категория, техническая характеристика, складской остаток, конфигурация, корзина, заказ, позиция заказа, платеж, задание на сборку, отгрузка, уведомление и журнал событий. Для конфигурации важны совместимость процессора и материнской платы, тип памяти, форм-фактор корпуса, длина видеокарты, мощность и разъемы блока питания.

Бизнес-процесс включает следующие основные шаги: просмотр каталога; подбор компонентов; проверка совместимости; добавление товаров в корзину; оформление заказа с повторной проверкой цены и остатка; резервирование; оплата через внешнюю платежную систему; комплектация; сборка и тестирование; передача в доставку или выдача; завершение заказа. Отмена и возврат выполняются с освобождением резерва или возвратом оплаты.

Границы системы: клиентский каталог и конфигуратор, корзина, оформление и оплата заказа, складской учет, задания на сборку и тестирование, доставка, уведомления, административное управление каталогом и пользователями. Вне границ остаются бухгалтерский учет, закупки у поставщиков, реальные банковские интеграции и производственный учет за пределами сборочного участка.

Действующие лица и их интересы:

– покупатель — выбрать совместимые компоненты, оформить и оплатить заказ, видеть актуальный статус;

– менеджер — управлять каталогом, ценами, активностью товаров и обрабатывать заказы;

– сотрудник склада — видеть состав заказа, комплектовать его, принимать приход, списание и возврат;

– сборщик — получать задание, отмечать начало и завершение сборки, фиксировать результат теста;

– администратор — управлять учетными записями, ролями и правами доступа.

1.2 Функциональные требования

Источником требований является спецификация, разработанная в лабораторной работе № 1. В таблице 2 приведены 22 функциональных требования и указано, где они реализованы и проверены. Сокращения «API» и «SPA» раскрыты в перечне сокращений.

Таблица 2 – Функциональные требования и их реализация

ID	Требование	Реализация и проверка
FR-01	Каталог активных товаров с категорией, ценой, характеристиками и наличием	GET /api/products; SPA-каталог; Playwright
FR-02	Поиск по наименованию, фильтры по категории и характеристикам	Параметры q, category, spec; SPA; Playwright
FR-03	Карточка товара с описанием, ценой и доступностью	GET /api/products/{id}; карточка SPA
FR-04	Конфигуратор с добавлением и изменением компонентов	SPA «Собрать ПК»; PUT /api/configuration
FR-05	Проверка совместимости и показ конфликтов	domain.Compatibility; POST /api/compatibility; тест и Playwright
FR-06	Корзина: добавление, изменение количества, удаление	GET/PUT /api/cart; SPA-корзина
FR-07	Оформление заказа с контактами, способом получения, повторной проверкой	POST /api/orders; проверка цены, остатка и совместимости
FR-08	Фиксация цены каждой позиции при создании заказа	order_items.price; снимок названия и характеристик
FR-09	Создание заказа и резервирование в одной транзакции с откатом	pgx.Tx, FOR UPDATE, stock; тест конкурентного резервирования
FR-10	Создание платежной операции и прием подтверждения провайдера	POST /api/orders/{id}/payment; webhook с HMAC
FR-11	Идемпотентность повторных платежных событий	payment_events.event_id PRIMARY KEY; тест повтора
FR-12	Отмена заказа с освобождением резерва или возвратом	POST /api/orders/{id}/cancel; release/refund; тест
FR-13	Задание на сборку для оплаченного заказа с услугой сборки	assembly_tasks; создание при оплате; Playwright
FR-14	Комплектация: состав заказа и отметка факта	orderAction pick/picked; склад; Playwright
FR-15	Сборка, тестирование и доработка	orderAction assemble/test/pass/fail; Playwright
FR-16	Хранение и показ статусов и времени переходов	orders.status; order_events; история в SPA
FR-17	Заявка внешней доставке, идентификатор отправления, статус получения	POST /shipment; syncShipment; Playwright
FR-18	Уведомления при ключевых изменениях статуса	notifications; создание в event(); API
FR-19	Управление товарами, категориями, ценами и активностью	POST /api/staff/products; /staff/categories; роли manager/admin
FR-20	Приход, списание и возврат товара без отрицательного остатка	POST /api/inventory/movement; stock_movements; SQL-проверка

ID	Требование	Реализация и проверка
FR-21	История статусов, платежей и сборки заказа	GET /api/orders/{id}; история и связанные записи
FR-22	Регистрация, вход, управление пользователями и ролями, обновление токенов	auth/register, login, refresh; users; bcrypt, JWT

Требования FR-01–FR-22 покрывают клиентские, складские, сборочные, платёжные и административные сценарии. Для каждого требования в репозитории есть программный компонент; для критичных сценариев добавлены автоматические проверки. Полнота покрытия ограничена учебным характером внешних интеграций: платежная и доставочная системы заменены локальным симулятором.

1.3 Нефункциональные требования и ограничения

Нефункциональные требования определяют качество решения и условия эксплуатации. Они сформулированы в лабораторной работе № 1 и приведены в таблице 3 вместе с фактическим состоянием проверки.

Таблица 3 – Нефункциональные требования и состояние проверки

ID	Требование	Состояние
NFR-01	Производительность: 95 % запросов каталога не более 2 с при 100 сессиях	Целевой ориентир; нагрузочный тест 100 сессий не выполнялся
NFR-02	Целостность: остатки, резервы и статусы изменяются транзакционно	Выполнено: транзакции PostgreSQL, ограничения CHECK, тесты
NFR-03	Надежность: подтвержденный заказ не теряется при перезапуске сервера	Выполнено на уровне СУБД; испытание перезапуском не оформлялось
NFR-04	Безопасность: HTTPS, хранение паролей только в виде стойких хэшей	bcrypt; TLS-конфигурация nginx; проверка HTTPS в docker-стеке не выполнялась
NFR-05	Авторизация: проверка роли и полномочий в каждом защищенном методе	Выполнено: middleware auth, allowed(), тест доступа
NFR-06	Удобство: понятные сообщения при конфликте и отсутствии товара	Выполнено: текстовые ошибки API и интерфейса, Playwright
NFR-07	Интерфейс от 360 px; проверка Chromium, Firefox, WebKit	Выполнено: 360 px, три движка, 36 проверок
NFR-08	Масштабируемость: несколько экземпляров без критичного состояния в памяти	Архитектура stateless API; нагрузочная проверка не выполнялась
NFR-09	Резервное копирование БД не реже раза в сутки и восстановление	Сервис backup с pg_dump; восстановление не проверялось
NFR-10	Наблюдаемость: журналы с request_id без секретов	Выполнено: slog, request_id, Cache-Control
NFR-11	Конфигурируемость: адреса, параметры и секреты задаются средой	Выполнено: переменные окружения, .env.app, .env.test

Ограничения и бизнес-правила, существенные для реализации: цена позиции фиксируется в заказе; доступный и зарезервированный остатки неотрицательны; конкурентные заказы не могут зарезервировать одну единицу товара; конфигурация не подтверждается при нарушении обязательного правила совместимости; задание на сборку создается только для оплаченного заказа с услугой сборки; отмена после начала сборки

доступна только уполномоченному сотруднику; внешние идентификаторы платежа и доставки уникальны.

1.4 Постановка задачи

Задача этапа I — создать воспроизводимую среду разработки и организовать процесс: развернуть ВМ, установить Redmine и Git, создать проект, участников, роли, трекеры, статусы и переходы, построить дерево целей и задач, продемонстрировать модель Git Flow. Результат этапа фиксируется в отчете снимками экрана и командами.

Задача этапа II — реализовать информационную систему, удовлетворяющую требованиям FR-01–FR-22, и проверить критические сценарии. Выбрана модульная монолитная архитектура: Go-сервер с REST API, SPA на React/TypeScript, PostgreSQL и отдельный учебный сервис-симулятор платежей и доставки. Такое решение сохраняет простоту развертывания и транзакционную целостность, достаточные для учебного проекта.

Критерии приемки: каталог и конфигуратор работают; конфликт совместимости обнаруживается; заказ создается и резервирует остатки атомарно; цена фиксируется; повторное платежное событие не проводится дважды; сборка, тестирование, доработка, доставка и завершение заказа отражаются в истории; интерфейс работает при ширине 360 px в трех браузерных движках; модель работ Redmine содержит 38 задач и три участника; Git-история содержит осмысленные коммиты и слияние.

2 Этап I. Организация технологического процесса

2.1 Виртуальная машина и среда разработки

Для изоляции сервисов от хостовой системы развернута виртуальная машина QEMU/KVM с Ubuntu Server 24.04 LTS Minimal. Виртуальная машина работает с аппаратным ускорением KVM, имеет два виртуальных процессора, 1536 МБ оперативной памяти и динамический диск QCOW2 емкостью 16 ГБ. Графический рабочий стол не используется: управление выполняется по SSH, Redmine доступен через браузер хоста. Параметры среды приведены в таблице 4.

Таблица 4 – Параметры виртуальной машины

Параметр	Значение
Гипервизор	QEMU/KVM, машина q35, accel=kvm, CPU host
Гостевая ОС	Ubuntu Server 24.04 LTS Minimal (24.04.5)
Виртуальные процессоры	2
Оперативная память	1536 МБ (регулируется VM_MEMORY_MB)
Диск	QCOW2 16 ГБ, backing file — официальный образ Ubuntu
Сеть	Проброс на loopback: HTTP 8082, SSH 2222
Redmine	6.1.5 из исходников, Ruby 3.2.3, Puma
СУБД	PostgreSQL 16.15
Web-сервер	nginx 1.24
Git	2.43
Тема	Bleuclair, русский язык интерфейса

Развертывание автоматизировано командами make vm-up, make vm-status, make vm-ssh, make vm-logs и make vm-down. При первом запуске скачивается официальный образ Ubuntu, проверяется его контрольная сумма, создается SSH-ключ и конфигурационный диск cloud-init. Установка Redmine выполняется внутри гостевой ОС; прогресс доступен в журнале, готовность подтверждается проверкой HTTP и входом администратора.

Виртуальная машина содержит отдельные каталоги данных и не использует Docker-экземпляры хоста. Диск и файлы Redmine сохраняются между запусками; для резервной копии предусмотрено штатное выключение и копирование каталога .vm. Такой подход позволяет повторно развернуть среду на другом компьютере, не затрагивая рабочие данные хоста.

Среда этапа II разворачивалась отдельно от Redmine: API на порту 8090, учебный провайдер на 8091, dev-сервер Vite на 5173 и тестовая база

PostgreSQL на 55434. Контейнерная конфигурация приложения собирается в образы opi34-app:local и opi34-web:local; она описана в разделе 3.4.

2.2 Установка и настройка Redmine

Redmine установлен из исходников внутри гостевой ОС. Порядок установки: подготовка пакетов и Ruby; создание роли и базы PostgreSQL; получение исходников Redmine 6.1.5; настройка database.yml и файла окружения; установка зависимостей bundle; генерация секретного ключа; миграции и загрузка начальных данных; настройка nginx и Puma; запуск служб и проверка входа.

Для раздачи статических файлов и проксирования запросов используется nginx, приложение запускается под Puma. Сервисы включаются в автозапуск. Пароль встроенной учетной записи admin генерируется случайно и сохраняется только в локальном файле .vm/credentials.txt; в репозиторий и отчет он не включается. Тема Bleuclair установлена в каталог тем Redmine и включена в настройках оформления.

Проверка установки включает версии Ruby, PostgreSQL, nginx и Git, состояние служб, доступность HTTP-страницы и вход пользователя с административными правами. Дополнительно проверяется, что обычный участник проекта не получает доступ к системной административной панели: для роли специалиста по ИС ответ /admin должен быть 403. Этот сценарий включен в автоматическую проверку Redmine.

После установки создан закрытый проект pc-assembly-34 с описанием варианта 34. Проект не является публичным; доступ предоставляется только участникам. В проекте включены модули задач, времени, новостей, документов, файлов, wiki, репозитория, календаря и диаграммы Ганта. Настройки проекта и участников приведены на рисунках 1 и 2.

2.3 Пользователи, роли и права

В Redmine созданы три участника команды и используется встроенная служебная учетная запись администратора. Для участников заданы

технические адреса в домене .invalid: отправка почты в учебной среде отключена, поэтому адреса не могут привести к реальным получателям. Пароли сгенерированы случайно, сохранены в локальном файле .vm/team-credentials.json и не публикуются. Состав учетных записей приведен в таблице 5.

Таблица 5 – Учетные записи Redmine

Логин	ФИО	Роль в проекте	Администратор Redmine
kochnev-so	Кочнев С.О.	Руководитель проекта	Да
kochnev-do	Кочнев Д.О.	Специалист по ИС	Нет
korolev-im	Королёв И.М.	DevOps-инженер	Нет
admin	Служебная учетная запись	—	Да

Ролевая модель построена по принципу минимально необходимых прав. Базовый набор разрешений для всех участников включает просмотр и редактирование задач, добавление комментариев, просмотр приватных заметок, сохранение запросов, диаграмму Ганта, календарь, учет времени, wiki, файлы, репозиторий и изменения. Руководитель дополнительно управляет проектом, участниками, версиями, категориями, связями задач и подзадачами, а также закрывает, возвращает и отменяет задачи. Специалист по ИС и DevOps-инженер дополнительно управляют подзадачами. Ключевые различия приведены в таблице 6.

Таблица 6 – Роли и ключевые права Redmine

Роль	Ключевые дополнительные права	Назначение
Руководитель проекта	Управление проектом, участниками, версиями, категориями, связями задач; закрытие, возврат и отмена задач	Планирование и приемка работ
Специалист по ИС	Управление подзадачами; базовые права на задачи, wiki, файлы и репозиторий	Установка, настройка и сопровождение ИС
DevOps-инженер	Управление подзадачами; базовые права на задачи, wiki, файлы и репозиторий	Среда, Git, развертывание и Git Flow

Права проверены практически: руководитель входит как администратор проекта и видит настройки; специалист по ИС входит как обычный пользователь и получает отказ при обращении к системной административной панели. Это подтверждает, что административные полномочия не выданы всем участникам автоматически.

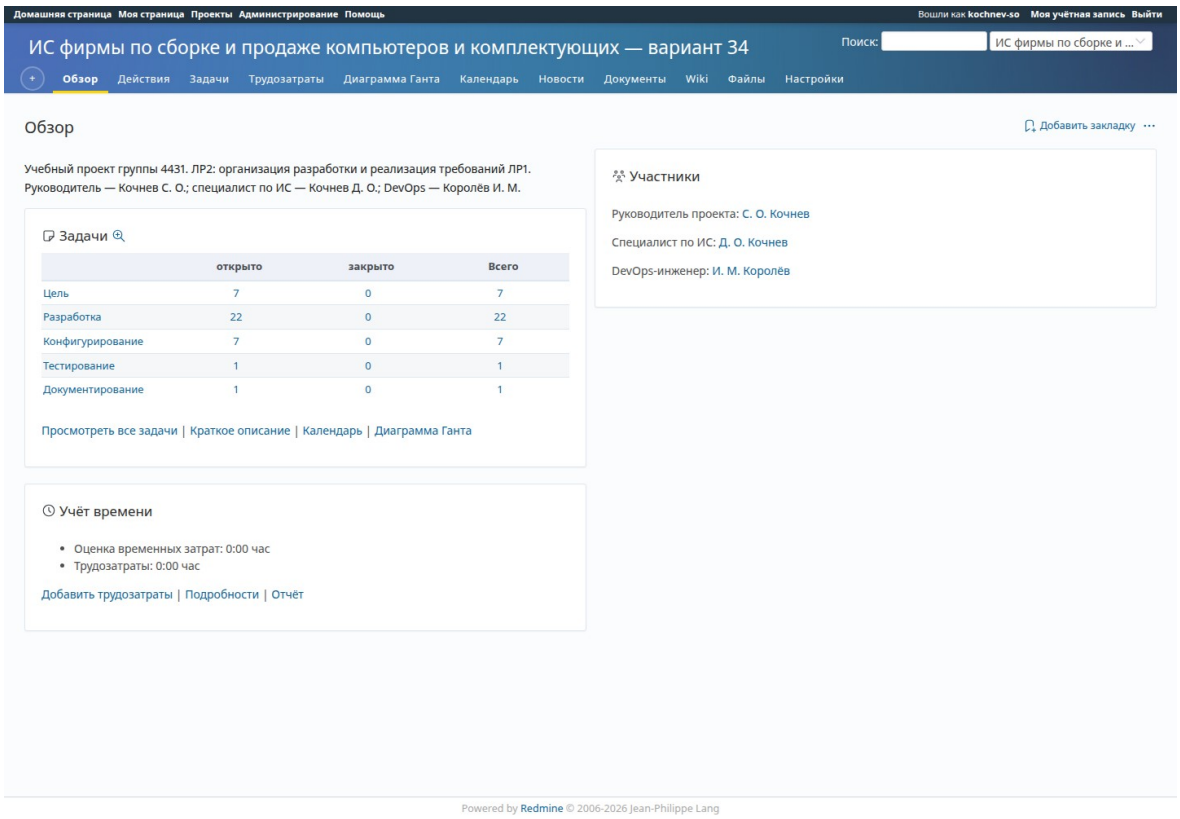


Рисунок 1 – Страница проекта pc-assembly-34 в Redmine

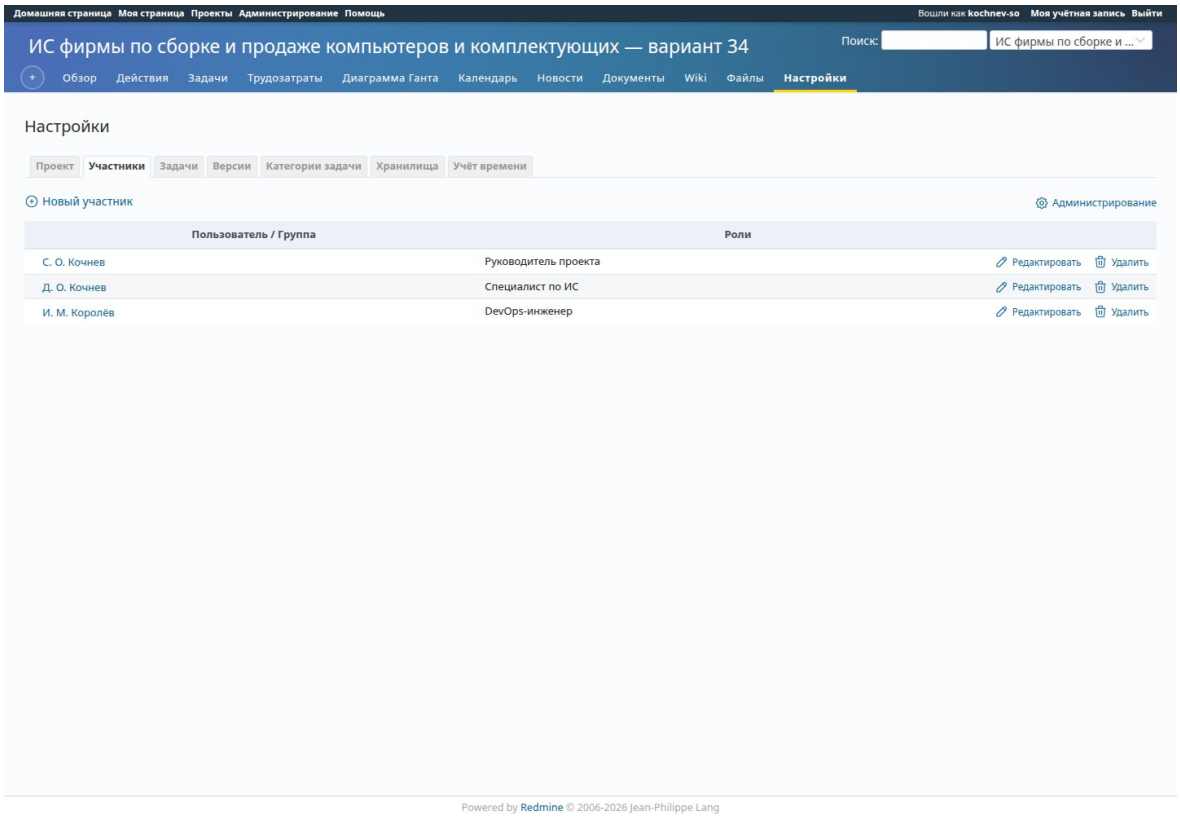


Рисунок 2 – Участники проекта и назначенные роли

2.4 Трекеры, статусы и переходы

Для проекта определены пять трекеров и семь статусов. Трекеры разделяют цели, разработку, конфигурирование, тестирование и

документирование; статусы образуют жизненный цикл задачи от планирования до закрытия или отмены. Состав трекеров и статусов приведен в таблице 7.

Таблица 7 – Статусы и трекеры проекта

Тип	Значение	Назначение
Статус	Запланирована	Задача создана, исполнитель еще не приступил
Статус	Назначена	Исполнитель определен, работа не начата
Статус	В работе	Работа выполняется
Статус	На контроле	Результат передан на проверку руководителю
Статус	Возвращена	Требуется доработка после проверки
Статус	Закрыта	Работа принята
Статус	Отменена	Работа не выполняется
Трекер	Цель	Крупные этапы и группы требований
Трекер	Разработка	Реализация функциональных требований
Трекер	Конфигурирование	Настройка среды, Git и инфраструктуры
Трекер	Тестирование	Проверки и приемочные испытания
Трекер	Документирование	Отчет, README и эксплуатационная документация

Переходы назначены ролям и трекерам. Базовый маршрут един для всех участников: «Запланирована → Назначена → В работе → На контроле», а также «Возвращена → В работе». Руководитель дополнительно переводит задачу из «На контроле» в «Закрыта» или «Возвращена», возвращает закрытую задачу и отменяет любую незакрытую задачу. Разрешенные переходы приведены в таблице 8.

Таблица 8 – Разрешенные переходы статусов

Исходный статус	Целевой статус	Кто выполняет
Запланирована	Назначена	Все роли
Назначена	В работе	Все роли
В работе	На контроле	Все роли
Возвращена	В работе	Все роли
На контроле	Закрыта	Руководитель
На контроле	Возвращена	Руководитель
Закрыта	Возвращена	Руководитель
Любой незакрытый статус	Отменена	Руководитель

Переходы применяются к каждому трекеру. Такая матрица не позволяет исполнителю самостоятельно закрыть задачу: закрытие и отмена остаются у руководителя, а возврат в работу доступен после замечаний. Это соответствует разделению ответственности, принятому в проекте.

2.5 Дерево целей и задач

Дерево целей и задач построено по требованиям первой лабораторной работы. Верхний уровень образуют два этапа: организация разработки и реализация информационной системы. В этап I входят семь задач по виртуальной машине, Redmine, проекту, ролям, дереву задач, Git и отчету. В этап II входят пять групп требований и 22 задачи FR-01–FR-22, а также две задачи на приемочные испытания и документацию. Итого в проекте 38 задач.

Таблица 9 – Состав дерева целей и задач

Уровень дерева	Количество	Примеры
Этап I — организация разработки	1	Цель верхнего уровня
Задачи этапа I	7	ВМ; проект и роли; дерево задач; Redmine; Git Flow; репозиторий; отчет
Этап II — реализация ИС	1	Цель верхнего уровня
Группы требований	5	Каталог; заказ; оплата; склад и сборка; пользователи
Задачи FR-01–FR-22	22	Каталог, совместимость, оплата, сборка, доставка и др.
Приемка и документация	2	Приемочные испытания; документация запуска
Итого	38	2 + 7 + 5 + 22 + 2

Задачи связаны отношениями «родительская — дочерняя». Для каждой задачи указаны трекер, статус, приоритет, исполнитель и описание с критерием приемки. Исполнители этапа I назначены; задачи этапа II в исходной модели были оставлены без исполнителей и распределены в ходе реализации. Список 38 задач и назначения приведены на рисунке 3.

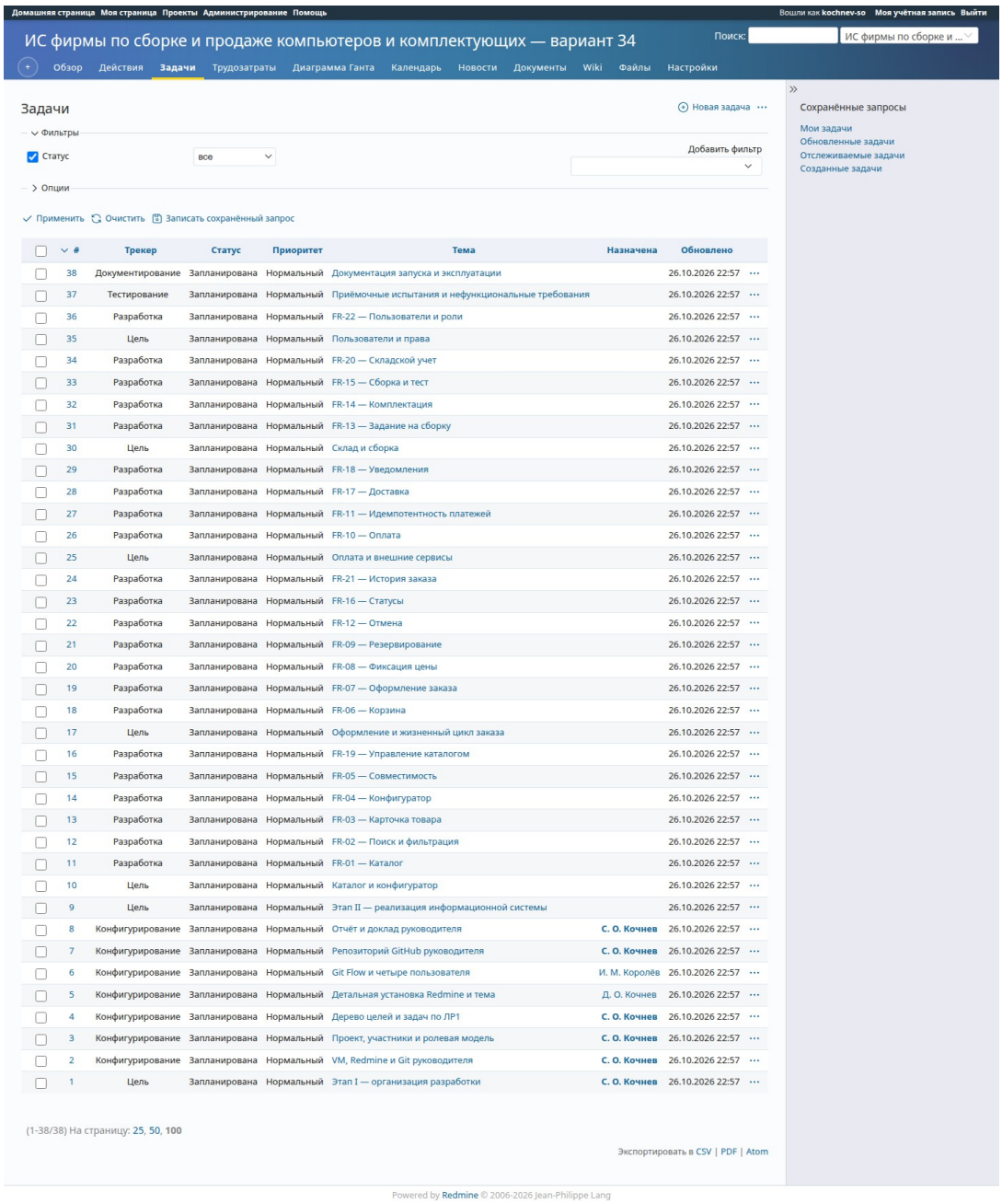


Рисунок 3 – Дерево из 38 задач проекта pc-assembly-34

На снимках видно, что проект закрытый, в списке участников три человека, а в списке задач присутствуют все 38 записей. Для задач этапа I назначены руководитель, специалист по ИС и DevOps-инженер; задачи этапа II сгруппированы по требованиям и связаны с целями.

2.6 Репозиторий и Git в виртуальной машине

В гостевой ОС установлен Git 2.43, создан учебный пользователь и настроены параметры авторства. Репозиторий проекта получен с GitHub, в него добавлены README.md и исходные файлы. Для этапа I достаточно

было условных исходников; в ходе этапа II в репозиторий добавлена реальная реализация, поэтому история содержит как инфраструктурные коммиты, так и коммиты приложения.

В виртуальной машине проверены базовые операции: клонирование, создание ветки, фиксация изменений, слияние и просмотр истории. Параметры авторства в ВМ являются учебными и не совпадают с учетной записью GitHub хоста; это сделано намеренно, чтобы не копировать ключи и токены хоста в гостевую среду. Отправка изменений в удаленный репозиторий выполняется с хоста после проверки.

Репозиторий используется как единое хранилище исходного кода, конфигурации развертывания, миграций, тестов и документации. В нем размещены каталоги cmd, internal, migrations, web, deploy, lab и docs. Отдельно описаны команды запуска ВМ и приложения, что позволяет воспроизвести результаты лабораторной работы.

3 Этап II. Реализация программного обеспечения

3.1 Архитектура и технологический стек

Реализация выполнена как модульный монолит. Серверное приложение на Go предоставляет REST API, работает с PostgreSQL и вызывает внешний учебный сервис платежей и доставки. Клиентская часть — одностраничное приложение React/TypeScript, собираемое Vite. В продуктивной конфигурации nginx принимает HTTPS-соединения, раздает статические файлы и проксирует запросы /api на серверное приложение. Схема взаимодействия компонентов приведена на рисунке 4.

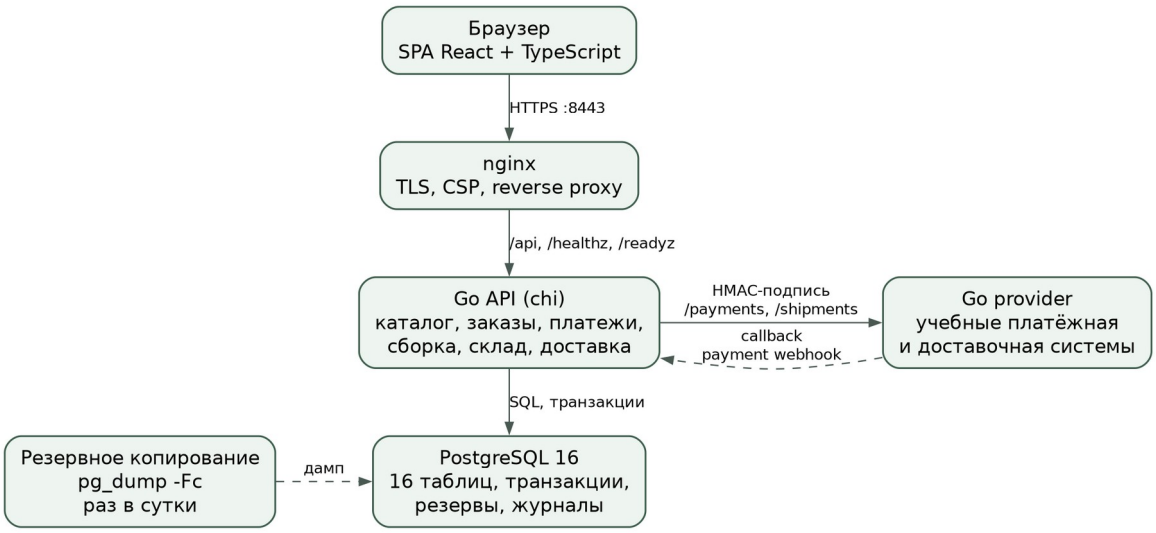


Рисунок 4 – Архитектура информационной системы

Модульный монолит выбран потому, что основные бизнес-операции требуют единой транзакции: создание заказа, резервирование остатков, фиксация цены и запись истории должны выполняться атомарно. Выделение этих операций в отдельные сервисы усложнило бы согласованность и увеличило бы объем инфраструктуры без практической пользы для учебного проекта. При этом границы пакетов позволяют позже выделить каталог, платежи или сборку в самостоятельные сервисы.

Таблица 10 – Технологический стек реализации

Компонент	Технологии	Назначение
Сервер приложения	Go 1.25, chi, pgx/v5, validator	REST API, бизнес-логика, транзакции
Аутентификация	JWT HS256, bcrypt, refresh-токены	Вход, обновление сессии, роли
База данных	PostgreSQL 16	Каталог, заказы, платежи, склад, задания
Клиент	React 19, TypeScript 5.8, Vite 7	Каталог, конфигуратор, корзина, заказы

Компонент	Технологии	Назначение
Внешний сервис	Go-симулятор платежей и доставки	Учебные интеграции и callback
Web-сервер	nginx 1.28 Alpine	TLS, статика, reverse proxy, CSP
Контейнеры	Docker, Docker Compose	Сборка и запуск стека
Тесты	Go testing, Playwright 1.62	Приемочные и кроссбраузерные проверки

Серверная часть разделена на пакеты internal/app и internal/domain. В internal/app находятся HTTP-обработчики, аутентификация, работа с заказами, каталогом, складом и интеграциями. В internal/domain сосредоточены модель предметной области, правила совместимости и переходы состояний. Такое разделение позволяет тестировать бизнес-правила отдельно от HTTP-слоя.

Клиентская часть содержит каталог с поиском и фильтрами, карточку товара, конфигуратор ПК, корзину, оформление заказа, страницу заказа с историей, экраны склада, сборщика, менеджера и администратора. Маршрутизация выполняется на клиенте; API возвращает JSON. Для повторного использования запросов реализован слой api.ts с обновлением access-токена при истечении сессии.

3.2 Модель данных

Модель данных включает 16 таблиц. Основные сущности — пользователи, товары, остатки, заказы, позиции заказов, платежи, задания на сборку, отгрузки, события и уведомления. Связи и ключевые атрибуты приведены на рисунке 5; назначение таблиц и ограничения — в таблице 11.

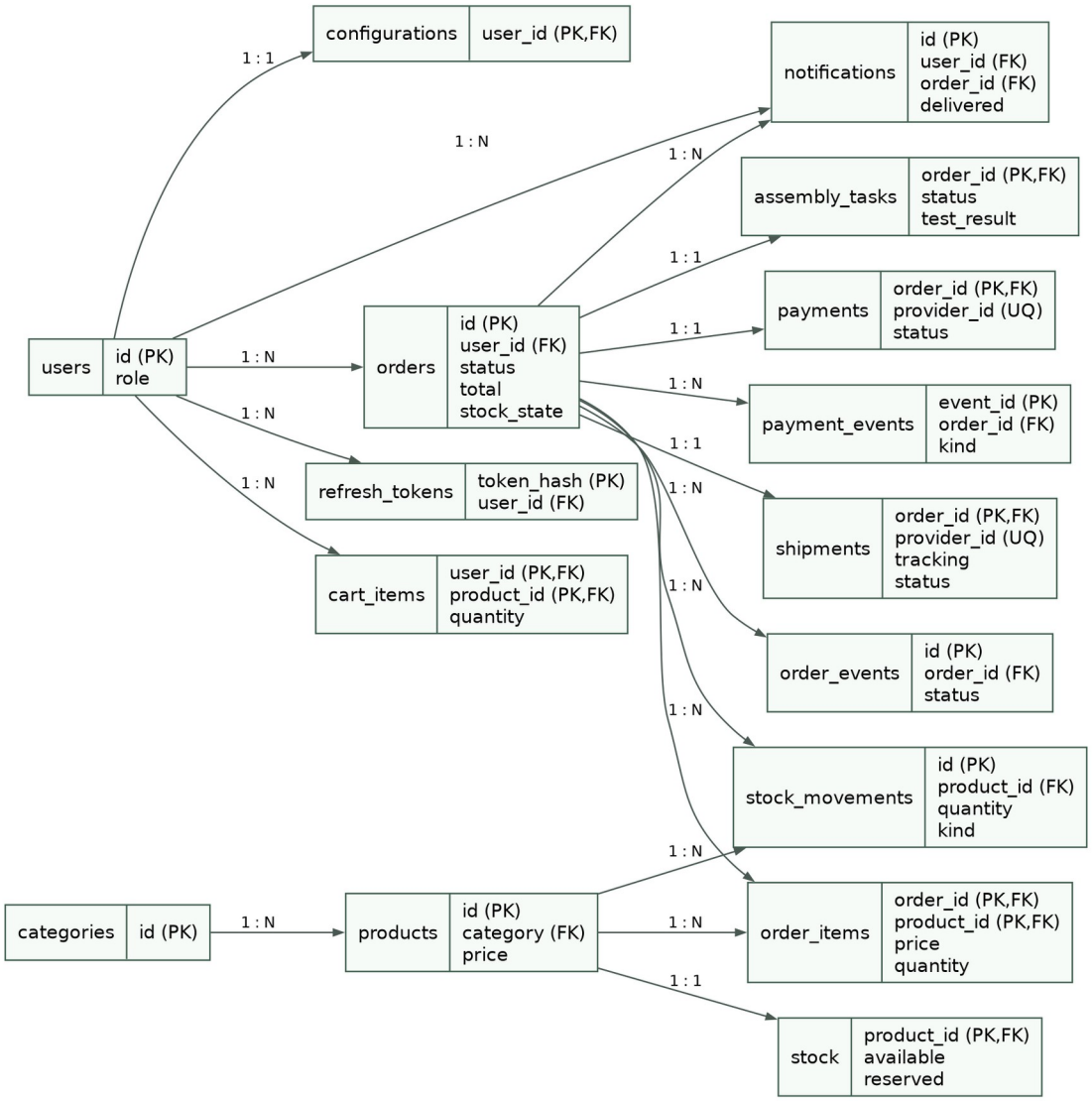


Рисунок 5 – Логическая модель данных PostgreSQL

Таблица 11 – Сущности базы данных

Таблица	Назначение	Ключевые ограничения
users	Учетные записи и роли	email UNIQUE; role CHECK; active
refresh_tokens	Хэши refresh-токенов	token_hash PK; срок действия
categories	Категории каталога	id PK
products	Товары и характеристики	price > 0; category FK; specs jsonb
stock	Доступный и зарезервированный остаток	available, reserved >= 0; product_id PK
cart_items	Корзина пользователя	quantity 1..100; составной PK
configurations	Сохраненная конфигурация ПК	user_id PK; items jsonb
orders	Заказы и состояние	status; total > 0; checkout_key; stock_state
order_items	Снимок состава и цен заказа	price > 0; quantity > 0; составной PK
order_events	История статусов	order_id FK; actor_id FK; created_at
assembly_tasks	Задание на сборку	order_id PK; status; test_result
payments	Платежная операция	order_id PK; provider_id UNIQUE; amount
payment_events	Идемпотентность событий	event_id PK; order_id FK; kind
shipments	Отгрузка и tracking	order_id PK; provider_id UNIQUE
stock_movements	Движения склада	quantity; kind; actor_id; order_id
notifications	Уведомления покупателю	user_id FK; order_id FK; delivered

Целостность обеспечивается первичными и внешними ключами, ограничениями CHECK и транзакциями. Цена в order_items сохраняется

отдельно от products.price, поэтому изменение каталога не меняет исторические заказы. Остаток хранится в двух полях: available и reserved; операции резервирования и освобождения изменяют оба поля согласованно. Для повторной обработки платежных событий используется первичный ключ event_id в payment_events.

История заказа не перезаписывается: каждое изменение статуса добавляет запись в order_events с автором и временем. Это позволяет восстановить последовательность действий и показать покупателю полную историю. Уведомления также создаются в транзакции изменения статуса, поэтому событие и уведомление не расходятся.

3.3 Бизнес-алгоритмы и жизненный цикл заказа

Проверка совместимости. Конфигуратор передает набор идентификаторов компонентов и признак полноты сборки. Сервер загружает активные товары и применяет правила, приведенные в таблице 12. Проверка выполняется как до добавления в корзину, так и повторно при оформлении заказа, поэтому изменение каталога между этими шагами не позволяет подтвердить несовместимую конфигурацию.

Таблица 12 – Правила совместимости компонентов

Правило	Проверка
Сокет процессора	cpu.socket должен совпадать с motherboard.socket
Тип памяти	memory_type каждой планки должен совпадать с memory_type платы
Форм-фактор	board_formats корпуса должен содержать form_factor платы
Длина видеокарты	gpu.length_mm не должна превышать case.gpu_max_mm
Мощность БП	power_w с запасом 20 % должен покрывать TDP процессора, видеокарты и 100 Вт
Разъемы БП	connectors должны содержать разъемы платы и видеокарты
Полнота сборки	обязательны CPU, плата, память, накопитель, БП и корпус; по одному на категорию, кроме памяти и накопителей

Оформление заказа. Клиент передает позиции, контакты, способ получения, признак сборки и уникальный checkout_key. Сервер в одной транзакции блокирует строку пользователя, проверяет повтор запроса по ключу, блокирует строки товаров и остатков, сверяет активность товара, ожидаемую цену и доступное количество, повторно проверяет совместимость, создает заказ и позиции, уменьшает available и увеличивает reserved, записывает движения склада, очищает корзину и добавляет событие

«Ожидает оплаты». При любой ошибке транзакция откатывается полностью. Повторный запрос с тем же checkout_key возвращает существующий заказ, не создавая дубль.

Оплата и идемпотентность. При создании платежа сервер создает запись payments и вызывает учебный провайдер, подписывая запрос HMAC-SHA256 с временной меткой. Провайдер возвращает идентификатор платежа; сервер сохраняет его. Подтверждение приходит на webhook, подпись и временная метка проверяются, сумма и идентификатор сверяются с заказом. Событие вставляется в payment_events с ON CONFLICT DO NOTHING; если событие уже обработано, бизнес-операция не повторяется. При оплате заказа с услугой сборки создается задание assembly_tasks.

Жизненный цикл заказа. Состояния и переходы приведены на рисунке 6. После оплаты заказ переходит в paid. Склад выполняет pick и picked; для заказа без сборки состояние сразу становится ready, для заказа со сборкой — picked. Сборщик переводит заказ в assembling, затем testing; успешный тест дает ready, неуспешный — rework с обязательной причиной. После повторной сборки и успешного теста заказ снова становится ready. Самовывоз завершает заказ действием pickup; для доставки создается shipping_pending, затем shipped и completed после подтверждения провайдера.

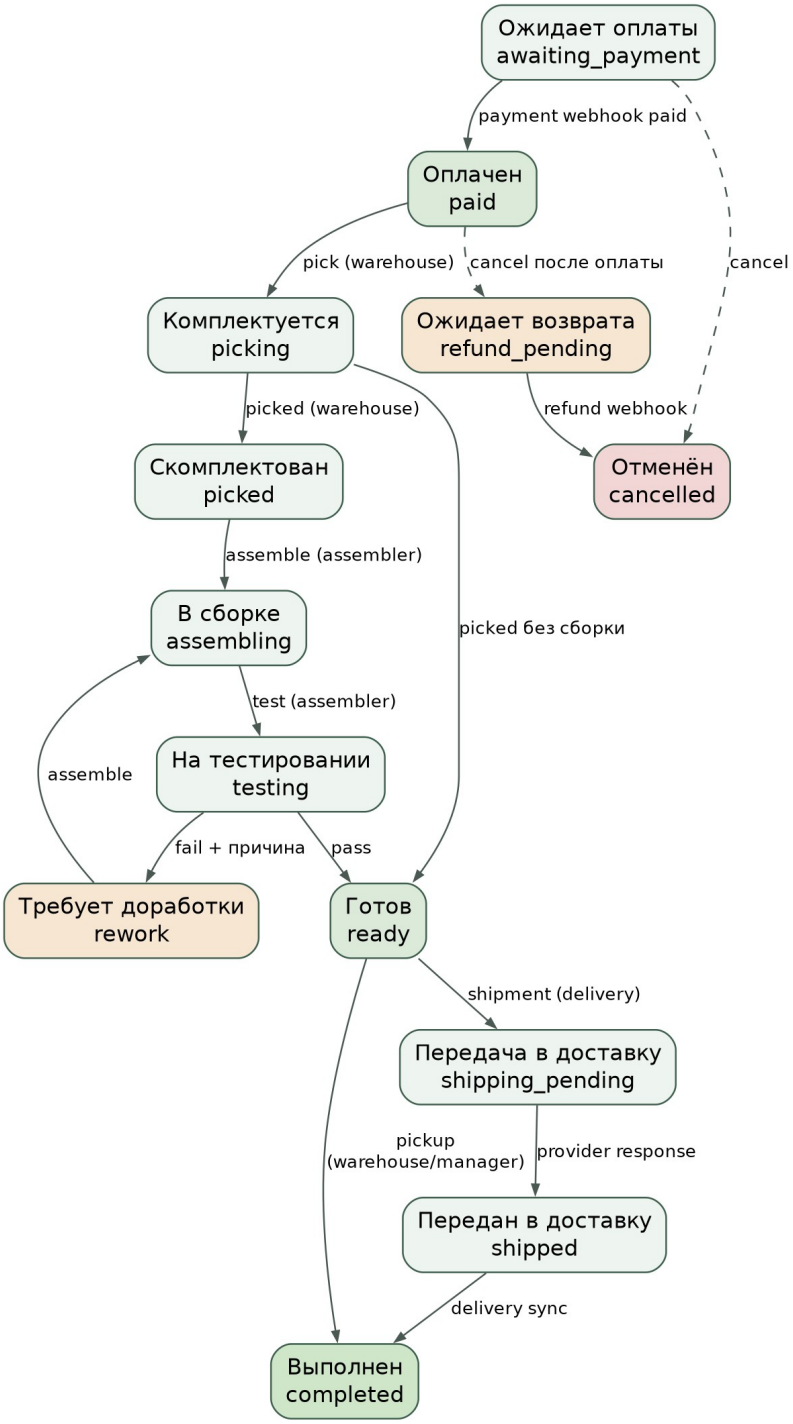


Рисунок 6 – Жизненный цикл заказа

Отмена. Покупатель или менеджер может отменить заказ до передачи в доставку. Если заказ еще не оплачен, резерв освобождается и статус становится cancelled. Если оплачен, создается refund_pending, освобождается резерв (или товар ожидает проверки склада), а после подтверждения возврата провайдером статус становится cancelled. Отмена после начала сборки

доступна только менеджеру; товар, уже переданный в работу, возвращается в доступный остаток только после проверки сотрудником склада.

Складской учет. Движения receipt, writeoff и return изменяют available в транзакции. SQL-условие available + delta >= 0 не позволяет уйти в отрицательный остаток; при нулевом числе обновленных строк возвращается конфликт. Каждое движение сохраняется в stock_movements с автором, основанием и, если применимо, идентификатором заказа. Возврат после отмены выполняется отдельным действием склада с обязательным примечанием.

Доставка и уведомления. Создание отгрузки переводит заказ в shipping_pending и вызывает провайдера; после успешного ответа сохраняются provider_id и tracking, статус становится shipped. Синхронизация статуса доставки вызывается отдельно; при delivered заказ становится completed. Каждое изменение статуса добавляет запись в order_events и уведомление в notifications, поэтому покупатель видит историю и может получить уведомление в интерфейсе.

Аутентификация и разграничение прав. Пароли хранятся в виде bcrypt-хэшей. Access-токен JWT подписывается HS256 и действует 15 минут; refresh-токен представляет собой случайную строку, хранится в базе только в виде SHA-256-хэша и действует 7 дней. При обновлении сессии старый refresh-токен удаляется, новый сохраняется; это обеспечивает ротацию. Cookie refresh-токена имеет атрибуты HttpOnly, Secure, SameSite=Strict. Каждый защищенный метод проверяет access-токен и роль; сервер дополнительно проверяет Origin запроса и возвращает 403 при недопустимом источнике.

3.4 Внешние интеграции, безопасность и развёртывание

Внешние платежная и доставочная системы заменены учебным сервисом cmd/provider. Он реализует создание платежа, подтверждение оплаты, создание отгрузки, изменение статуса доставки и подписывает ответы тем же секретом. Секрет передается через переменную

INTEGRATION_SECRET, длина не менее 32 символов. В учебном режиме доступны команды demo-payment и demo-delivery, которые позволяют пройти полный сценарий без реальных денег и внешних сетей. Это ограничение явно отражено в интерфейсе и отчете.

Безопасность реализована на нескольких уровнях: HTTPS и CSP в nginx; запрет X-Frame-Options DENY; X-Content-Type-Options nosniff; проверка Origin; ограничение размера JSON-тела; валидация входных данных; параметризованные SQL-запросы; bcrypt для паролей; HMAC и временное окно для webhook; ролевые проверки в обработчиках; отсутствие секретов в логах. Пароли и ключи хранятся в .env-файлах, которые исключены из Git; в репозитории присутствуют только примеры с пустыми значениями.

Развертывание описано в deploy/compose.app.yml. Сервисы db, migrate, backend, provider, frontend и backup запускаются в Docker Compose. Миграции и демонстрационные данные выполняются отдельным сервисом migrate до запуска backend. Для базы данных используется том app_data; для резервного копирования — сервис backup с pg_dump и ежедневным циклом. Frontend слушает 8443 (HTTPS) и 8083 (HTTP-редирект), backend и provider доступны только внутри сети Compose. Проверки healthcheck контролируют готовность базы, API и провайдера.

Локальная разработка использует отдельную тестовую базу на порту 55434 и dev-сервер Vite на 5173. Это позволяет запускать приемочные тесты, не затрагивая данные демонстрационного стека. Скрипт deploy/run-tests.sh читает .env.test, подставляет TEST_DATABASE_URL и запускает go test -race. Для сборки клиента используется npm ci и npm run build.

4 Тестирование и результаты

4.1 Методика и окружение

Проверки разделены на три уровня. Первый уровень — модульные и приемочные тесты Go, которые работают с отдельной базой PostgreSQL и проверяют транзакционные и конкурентные сценарии. Второй уровень — сквозные проверки интерфейса в Chromium, Firefox и WebKit через Playwright: каталог, поиск, мобильная ширина 360 px, вход, ротация refresh-токена, конфликт совместимости, оформление заказа, оплата, склад, сборка, доработка, доставка и завершение. Третий уровень — проверка модели работ Redmine: вход, закрытый проект, 38 задач, три участника и запрет административной панели для обычного пользователя.

Окружение проверок: API на http://127.0.0.1:8090, учебный провайдер на 8091, Vite на 5173, тестовая PostgreSQL на 55434. Приемочные тесты Go запускались командой make test (скрипт deploy/run-tests.sh) с флагом -race. Сборка клиента проверялась командой npm run build. Сквозные проверки выполнялись Playwright 1.62; для WebKit использовался контейнер mcr.microsoft.com/playwright:v1.62.1-noble, поскольку на хосте не хватало системных библиотек. Это ограничение среды, а не приложения.

Все проверки выполнялись на реальных данных, созданных в тестовой базе. Для каждого браузера создавался отдельный заказ, проходивший полный жизненный цикл; идентификаторы заказов сохранены в результатах. Скриншоты и JSON-файлы результатов приложены в каталоге docs/images и в отчете.

4.2 Модульные и приёмочные тесты

Приемочные тесты Go охватывают пять критичных сценариев. Каждый тест создает изолированную схему в базе с суффиксом _test и не влияет на другие запуски. Результаты приведены в таблице 13; общее время выполнения — 72,9 с, все тесты пройдены.

Таблица 13 – Приемочные тесты Go

Тест	Что проверяет	Результат
TestConcurrentLastItemReservation	Конкурентное резервирование последней единицы товара	Пройден
TestPriceSnapshotsAndAtomicRollback	Фиксация цен и полный откат при нехватке товара	Пройден
TestPaymentReplayAndAuthorization	Повтор платежного события и запрет доступа чужой роли	Пройден
TestAssemblyFailureRefundAndInspectedReturn	Неудачная сборка, возврат и проверенный возврат на склад	Пройден
TestCompatibilityRejectsConflict	Обнаружение конфликта совместимости	Пройден

Тест конкурентного резервирования подтверждает, что два одновременных заказа не могут получить одну и ту же единицу товара: второй запрос завершается конфликтом, а остаток остается неотрицательным. Тест фиксации цен проверяет, что после изменения каталожной цены существующий заказ сохраняет прежнюю цену, а при нехватке товара не создаются ни заказ, ни движения склада. Тест платежного повтора проверяет, что повторная доставка события не проводит оплату дважды и не создает повторное задание на сборку.

Тест сборки и возврата моделирует неуспешный тест, доработку, повторную сборку, отмену и проверенный возврат товара на склад. Тест совместимости проверяет конфликт сокета и корректную конфигурацию. Эти сценарии соответствуют наиболее рискованным бизнес-правилам и покрывают требования FR-05, FR-08, FR-09, FR-11, FR-12 и FR-15.

Сборка клиента выполнена командой npm run build: TypeScript-проверка и Vite собрали 96 модулей, итоговый JS-файл — около 351 КБ, CSS — около 13 КБ. Сборка завершилась без ошибок. Артефакты сборки исключены из Git.

4.3 Кроссбраузерные сценарии

Сквозные проверки выполнены в трех движках: Chromium 153.0.8010.47, Firefox 153.0 и WebKit 26.5. В каждом браузере выполнялись 12 проверок: каталог, поиск, адаптивная ширина 360 px, вход, ротация refresh-токена, конфликт совместимости, оформление заказа, оплата через учебный симулятор, склад, доработка сборки, доставка и выполненный заказ.

Всего выполнено 36 проверок, все пройдены. Результаты приведены в таблице 14.

Таблица 14 – Результаты кроссбраузерных проверок

Браузер	Версия	Номер заказа	Проверок	Результат
Chromium	153.0.8010.47	0a0f304c	12	Пройдено
Firefox	153.0	21ba1fbd	12	Пройдено
WebKit	26.5	d4fd858e	12	Пройдено

Каталог и поиск. Проверяется, что отображаются 13 демонстрационных товаров, поиск по слову «клавиатура» оставляет одну карточку, а очистка поиска возвращает полный список. На рисунке 7 показан каталог Chromium; на рисунке 11 — каталог Firefox и WebKit, подтверждающий единообразие отображения.

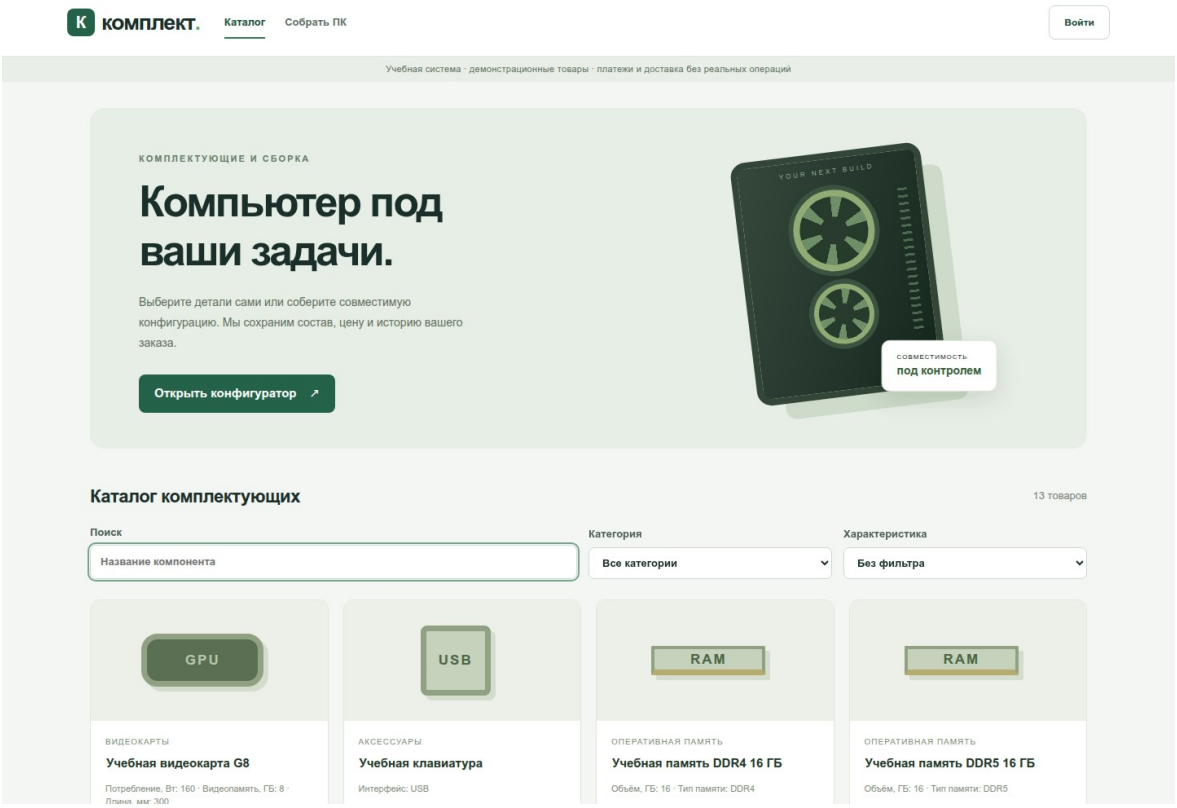


Рисунок 7 – Каталог товаров в Chromium

Конфигуратор и совместимость. Выбираются совместимые процессор, плата, память, накопитель, блок питания и корпус; появляется сообщение «Компоненты совместимы». Затем выбирается процессор с другим сокетом, и интерфейс показывает конфликт «Сокет процессора и платы не совпадает

или не указан». После возврата совместимого процессора конфликт исчезает. Снимок конфигуратора приведен на рисунке 8.

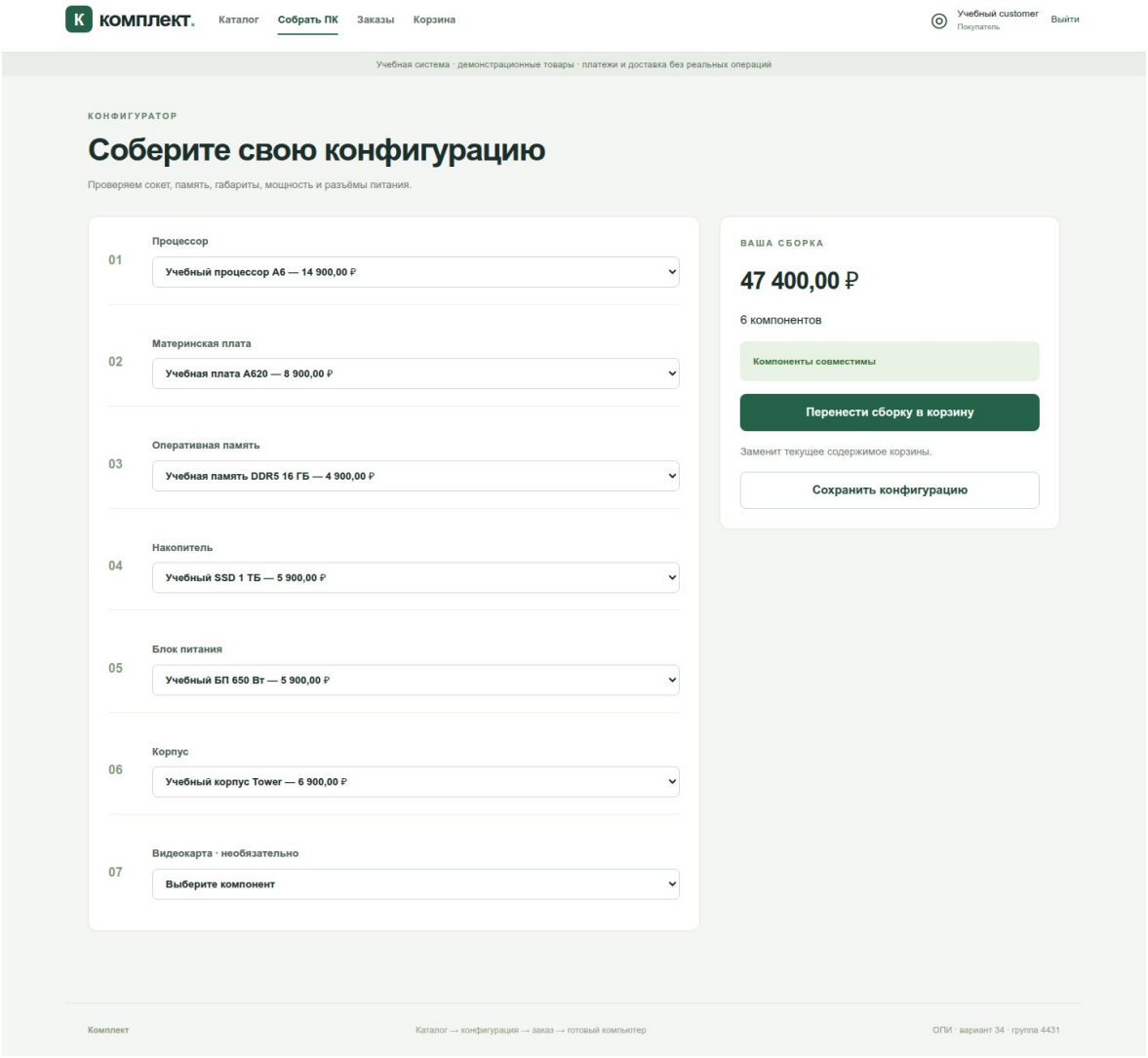


Рисунок 8 – Конфигуратор ПК и проверка совместимости

Мобильная версия. При ширине 360 px проверяется отсутствие горизонтальной прокрутки: ширина документа не превышает ширину окна. Полная страница каталога разбита на три вертикальные части на рисунке 9; это позволяет показать весь макет, не уменьшая текст до нечитаемого размера.

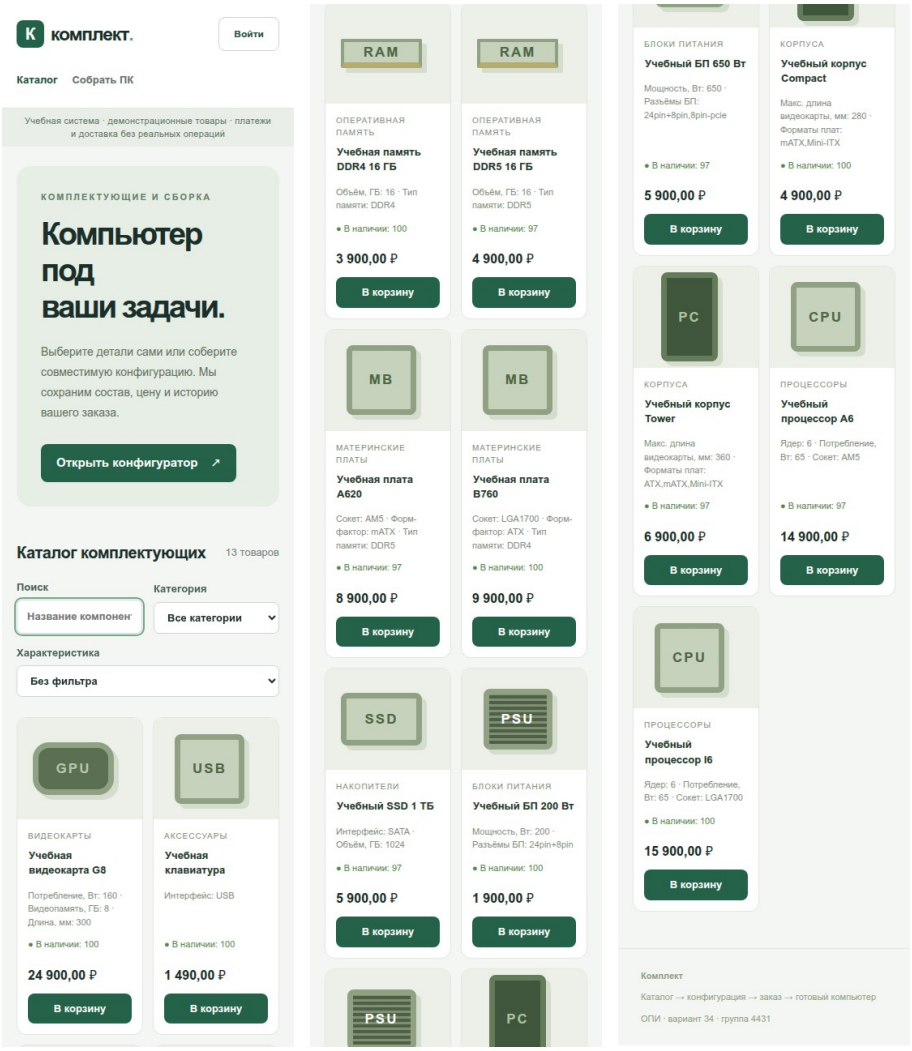


Рисунок 9 – Каталог при ширине экрана 360 px

Полный заказ. В каждом браузере создавался заказ со сборкой и доставкой, оплачивался через учебный провайдер, проходил комплектацию, сборку, неудачный тест, доработку, повторный тест, отгрузку и доставку. На рисунке 10 показан выполненный заказ: состав, зафиксированные цены, история статусов, задание на сборку, тест и оплата. Именно этот сценарий подтверждает требования FR-07–FR-18.

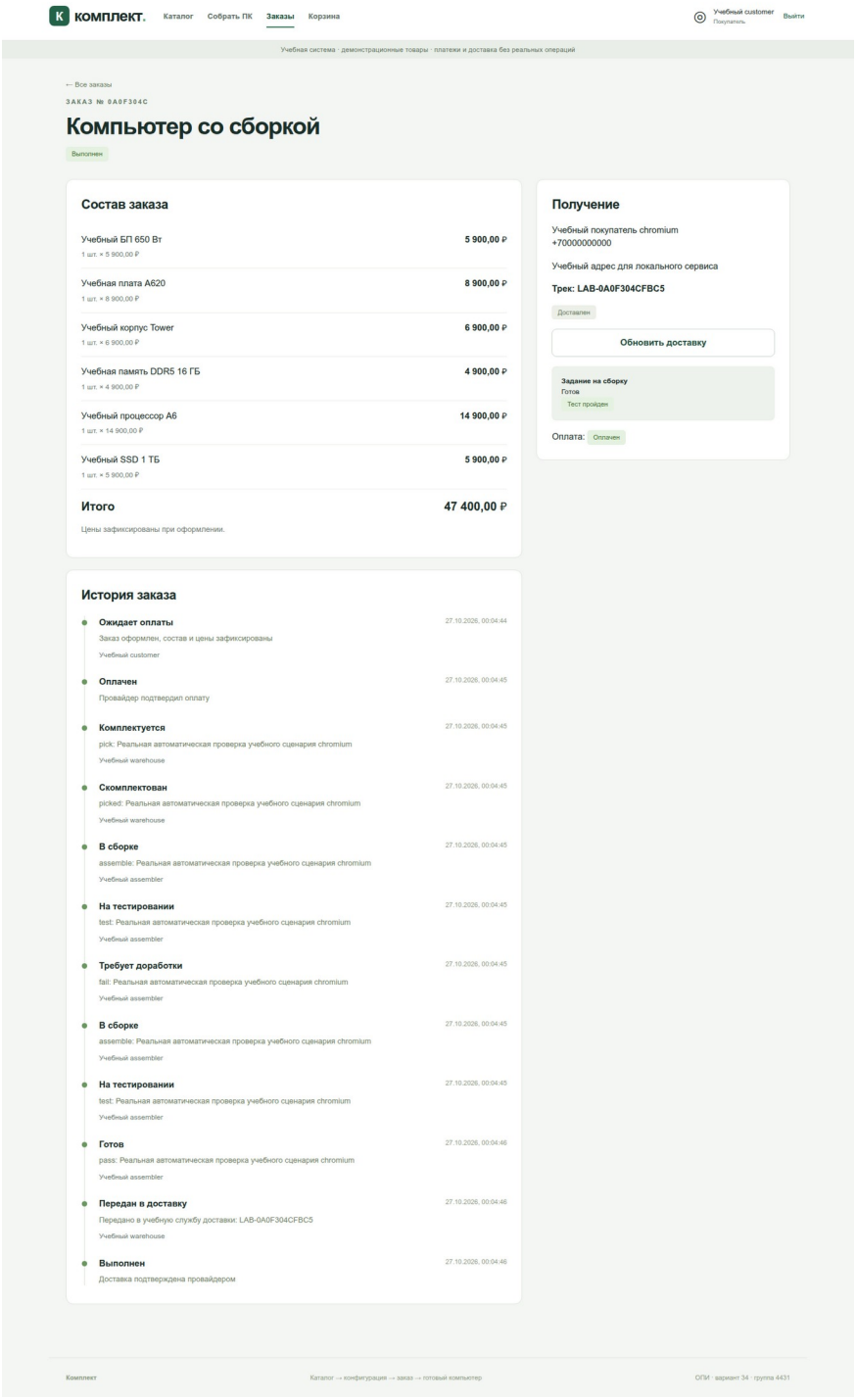


Рисунок 10 – Выполненный заказ с историей статусов

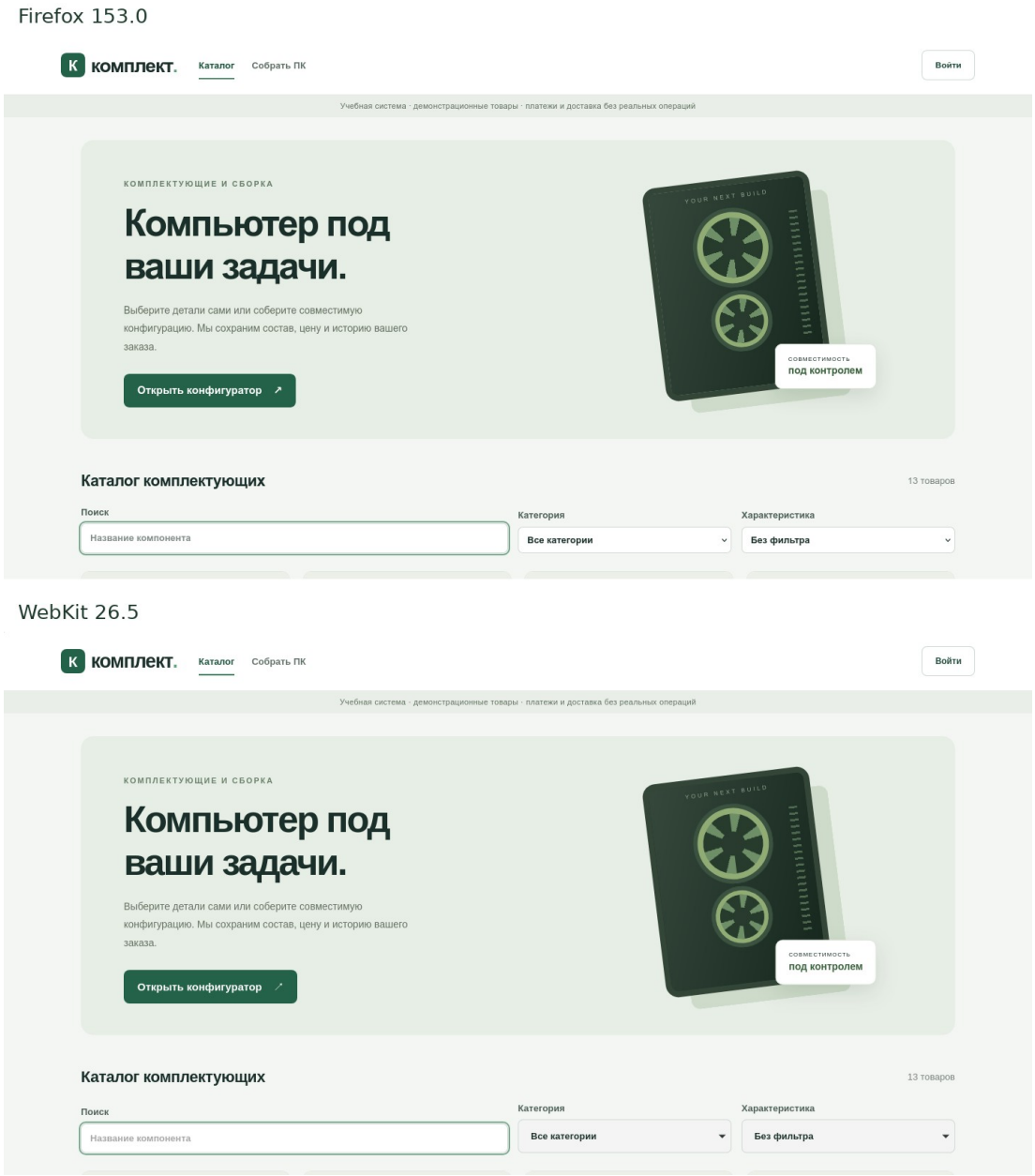


Рисунок 11 – Каталог в Firefox и WebKit

Ошибки интерфейса. В каждом браузере после завершения сценария проверялось отсутствие видимых сообщений role=alert. Ошибок не обнаружено. Это не заменяет полного набора UX-тестов, но подтверждает отсутствие явных сбоев в основном сценарии.

4.4 Проверка модели работ в Redmine

Проверка Redmine выполнялась через браузер и API. Использовался вход руководителя kochnev-so с административными правами; проверялись закрытый проект, список 38 задач, три участника и роли. Затем выполнялся вход специалиста kochnev-do и проверялось, что системная

административная панель недоступна: ответ /admin равен 403. Результаты приведены в таблице 15.

Таблица 15 – Проверки модели работ Redmine

Проверка	Ожидаемый результат	Фактический результат
Вход руководителя	Успешный вход, доступ к проекту	Пройдено
Закрытый проект	Проект не публичен, доступ только участникам	Пройдено
Количество задач	38 задач	38
Участники	Три участника с ролями	Три
Обычный пользователь и /admin	HTTP 403	403

Снимки проекта и участников приведены на рисунках 1 и 2, список задач — на рисунке 3. Проверка подтверждает, что модель работ не только описана в отчете, но и существует в работающей системе. Пароли и содержимое файлов с секретами в отчет не включены.

4.5 Соответствие нефункциональным требованиям

Соответствие нефункциональным требованиям оценивалось по фактически выполненным проверкам и по состоянию реализации. Часть требований подтверждена автоматическими тестами, часть — конфигурацией и архитектурными решениями, часть вынесена в ограничения. Сводная оценка приведена в таблице 16.

Таблица 16 – Соответствие нефункциональным требованиям

NFR	Фактическое подтверждение	Статус
NFR-01	Целевой ориентир по задержкам; нагрузка 100 сессий не измерялась	Не проверено
NFR-02	Транзакции, CHECK-ограничения, тесты резервирования и отката	Подтверждено
NFR-03	Данные хранятся в PostgreSQL; испытание перезапуском не оформлялось	Частично
NFR-04	bcrypt, TLS-конфигурация nginx, секреты вне Git	Частично
NFR-05	Middleware авторизации, ролевые проверки, тест 403	Подтверждено
NFR-06	Понятные ошибки API и интерфейса, сквозной сценарий	Подтверждено
NFR-07	360 px, Chromium/Firefox/WebKit, 36 проверок	Подтверждено
NFR-08	Stateless API; нагрузочная проверка не выполнялась	Частично
NFR-09	Сервис backup с pg_dump; восстановление не проверялось	Частично
NFR-10	slog, request_id, отсутствие секретов в логах	Подтверждено
NFR-11	Переменные окружения, .env-файлы, примеры без секретов	Подтверждено

Честная фиксация частично подтвержденных требований важна для защиты: отчет не заявляет нагрузочные испытания, восстановление из резервной копии и проверку HTTPS в docker-стеке как выполненные. Эти работы требуют отдельного стенда и вынесены в направления развития.

5 Применение Git и модели Git Flow

5.1 Ветвление и история репозитория

Проект размещен в репозитории GitHub Weit145/opi-suai. В истории использована модель Git Flow: функциональная ветка feature/init-skeleton была влита в develop; из develop создана ветка release-0.1.0, влитая в main и отмеченная тегом v0.1.0; исправление hotfix-0.1.1 влито в main и develop и отмечено тегом v0.1.1. Такая схема разделяет разработку, подготовку релиза и срочные исправления.

Для этапа II создана ветка feature/app-implementation. Она была создана от main, а не от develop, потому что удаленная ветка develop отстает от main и не содержит файлов виртуальной машины, Makefile и README этапа I. Создание ветки от develop привело бы к потере этих файлов при слиянии. Это отклонение от классического Git Flow, обусловленное состоянием учебного репозитория; оно зафиксировано в отчете.

В ветке feature/app-implementation выполнены пять осмысленных коммитов: серверная часть, клиентская часть, тесты и модель Redmine, контейнерное развертывание, документация. Затем ветка влита в main с флагом --no-ff, чтобы сохранить факт слияния, и отмечена тегом v0.2.0. История коммитов приведена в таблице 17.

Таблица 17 – Коммиты этапа II

Хэш	Тип	Сообщение	Содержание
6650311	feat(api)	implement PC shop order lifecycle	Go API, провайдер, схема, бизнес-правила
26844df	feat(web)	add React storefront for catalog and orders	SPA, каталог, конфигуратор, заказы
934cc32	test(lab)	cover order scenarios and Redmine setup	Приемочные тесты, проект Redmine
4c63062	build(deploy)	add container stack and test runner	Docker, Compose, nginx, Makefile
5f3896f	docs	describe the application and test workflow	README, запуск и тесты
a21d1d2	merge	Merge feature/app-implementation	Слияние в main, тег v0.2.0

Команды, использованные при интеграции: git switch -c feature/app-implementation; git add <файлы>; git commit -m <сообщение>; git switch main; git merge --no-ff feature/app-implementation; git tag -a v0.2.0 -m "Stage II application". Для проверки истории применялись git log --graph --oneline --all и

git status. Изменения зафиксированы локально; отправка в удаленный репозиторий выполняется отдельно после согласования.

5.2 Интеграция этапа II

Перед слиянием проверено, что в коммиты не попали секреты: .env.app, .env.test, каталог .vm, сертификаты deploy/certs и локальные артефакты .local исключены через .gitignore. В репозиторий добавлены только примеры конфигурации deploy/app.env.example и deploy/test.env.example с пустыми значениями. Это соответствует требованию NFR-11 и практике безопасной работы с Git.

Слияние выполнено с --no-ff, поэтому в истории виден отдельный merge-коммит. Тег v0.2.0 фиксирует состояние этапа II. После слияния рабочее дерево содержит приложение, миграции, тесты, конфигурацию развертывания, проект Redmine и отчет. Ветка feature/app-implementation сохранена для демонстрации; при необходимости ее можно удалить после публикации.

Модель Git Flow применена не формально, а с учетом реального состояния веток. Для учебного репозитория важно было сохранить результаты этапа I, поэтому ветка этапа II была создана от main. В дальнейшем при синхронизации develop с main можно вернуться к классической схеме: feature → develop → release → main, а срочные исправления выпускать через hotfix.

ВЫВОДЫ

1. Развернута воспроизводимая виртуальная среда QEMU/KVM с Ubuntu 24.04, Redmine 6.1.5, PostgreSQL 16, nginx, Puma и Git. Среда сохраняет данные между запусками и управляется командами make; это подтверждает готовность технологического процесса к повторению.

2. В Redmine создан закрытый проект pc-assembly-34, три участника, ролевая модель, пять трекеров, семь статусов и матрица переходов. Права проверены: руководитель имеет административные полномочия, обычный участник не получает доступ к системной панели.

3. Построено дерево из 38 задач: два этапа, семь задач этапа I, пять групп требований, 22 задачи FR-01–FR-22, приемочные испытания и документация. Дерево связано с требованиями первой лабораторной работы и отражает зоны ответственности.

4. Реализована информационная система: каталог, поиск, конфигуратор, проверка совместимости, корзина, оформление заказа, оплата, склад, сборка, тестирование, доработка, доставка, уведомления и администрирование. Серверная часть выполнена на Go, клиентская — на React/TypeScript, данные хранятся в PostgreSQL 16.

5. Обеспечена целостность критичных операций: создание заказа и резервирование выполняются в одной транзакции, цена позиции фиксируется, остатки не могут стать отрицательными, повторные платежные события не проводятся дважды. Эти свойства подтверждены приемочными тестами Go.

6. Проведены 36 кроссбраузерных проверок в Chromium, Firefox и WebKit, включая ширину 360 px, полный жизненный цикл заказа и оплату через учебный симулятор. Все проверки пройдены; результаты и снимки экрана приложены к отчету.

7. Настроен Git и продемонстрирована модель Git Flow: feature-ветка, пять осмысленных коммитов, слияние с --no-ff и тег v0.2.0. Секреты, локальные сертификаты и артефакты исключены из репозитория.

8. Зафиксированы ограничения: замена VirtualBox/Windows на QEMU/KVM/Ubuntu не согласована с преподавателем; платежная и доставочная системы являются учебным симулятором; нагрузочное тестирование 100 сессий, восстановление из резервной копии и проверка HTTPS в docker-стеке не выполнялись и требуют отдельной приемки.

9. Отчет оформлен по требованиям методических указаний: титульный лист, содержание, задание, описание хода работ, снимки экрана, выводы и список источников. Объем отчета превышает 15 страниц; каждый основной раздел начинается с новой страницы.

10. В результате работы сформированы практические навыки организации промышленного производства ПО: планирование работ, распределение ролей, управление проектом, контроль версий, разработка, тестирование и документирование. Полученные результаты могут быть использованы как основа для развития системы реальными интеграциями и нагрузочным тестированием.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Чакон С., Штрауб Б. Git для профессионального программиста. — СПб.: Питер, 2016. — 496 с.: ил. — (Серия «Библиотека программиста»). — ISBN 978-5-496-01763-3.

2. Система управления проектами Redmine : официальный сайт. — URL: https://www.redmine.org/ (дата обращения: 26.10.2026).

3. ГОСТ 2.105-2019. ЕСКД. Общие требования к текстовым документам. — М.: Стандартинформ, 2019.

4. ГОСТ 7.32-2017. СИБИД. Отчет о научно-исследовательской работе. Структура и правила оформления. — М.: Стандартинформ, 2017.

5. ГОСТ 7.0.100-2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — М.: Стандартинформ, 2018.

6. Документация языка Go : официальный сайт. — URL: https://go.dev/doc/ (дата обращения: 26.10.2026).

7. React : официальная документация. — URL: https://react.dev/ (дата обращения: 26.10.2026).

8. PostgreSQL 16 Documentation. — URL: https://www.postgresql.org/docs/16/ (дата обращения: 26.10.2026).

9. Playwright Documentation. — URL: https://playwright.dev/ (дата обращения: 26.10.2026).

10. Git Documentation. — URL: https://git-scm.com/doc (дата обращения: 26.10.2026).

ПРИЛОЖЕНИЕ А

Основные команды и структура проекта

Ниже приведены команды, использованные для развертывания, запуска и проверки. Команды выполняются из корня репозитория, если не указано иное.

```
make vm-up          # создать и запустить ВМ, установить Redmine
make vm-status      # проверить состояние ВМ и Redmine
make vm-ssh         # войти в гостевую Ubuntu по SSH
make vm-logs        # смотреть журнал установки
make vm-down        # штатно выключить ВМ

make app-init       # создать .env.app и локальные TLS-сертификаты
make app-up         # собрать и запустить docker-стек приложения
make app-down       # остановить стек
make app-logs       # журналы контейнеров

make test           # go test -race с тестовой БД из .env.test
cd web && npm ci && npm run build   # сборка клиента
```

Структура репозитория:

```
opi-suai/
  cmd/app/              HTTP API и миграции
  cmd/provider/         учебный платежный и доставочный сервис
  internal/app/         обработчики, заказы, интеграции, тесты
  internal/domain/      модель и правила совместимости
  migrations/           схема PostgreSQL
  web/                  React + TypeScript + Vite
  deploy/               Dockerfile, Compose, nginx, скрипты
  lab/                  требования и настройка Redmine
  vm/                   управление QEMU/KVM и провижининг
  docs/                 отчет, схемы и снимки экрана
```

Пример команд Git Flow, использованных при интеграции:

```
git switch -c feature/app-implementation
git add <файлы>
git commit -m "feat(api): implement PC shop order lifecycle"
git switch main
git merge --no-ff feature/app-implementation
git tag -a v0.2.0 -m "Stage II application"
git log --graph --oneline --all
```

Проверка секретов перед коммитом:

```
git status --short
git check-ignore -v .env.test .env.app .vm deploy/certs
git diff --cached --stat
```

Все команды и конфигурации проверены в ходе выполнения работы. Пароли, ключи и содержимое .env-файлов в приложении не публикуются.